Lab 5 Hands on Practice on Advanced JavaScript CLO-1 GA-4

Case Study

Air University is developing CampusFlow, a JavaScript-based academic operations system. Before the system is connected to a web interface, the development team wants the core business logic implemented and tested in Node.js. The system receives imperfect student data, performs academic analytics, manages different types of users, reads and validates JSON data, and communicates with asynchronous services. You have been assigned to implement the core JavaScript functionality.

General Requirements

1. Do not use DOM manipulation, HTML, React, Express, databases, or external packages.
2. Do not use closures as a solution technique. In particular, do not create functions that return inner functions for retaining state.
3. Use const and let appropriately. Avoid var unless specifically demonstrating its behavior.
4. Do not mutate source data unless the task explicitly requires mutation.
5. Do not hard-code calculated answers.
6. Functions should return meaningful results instead of only printing values.
7. Use strict equality (=== / !==) where appropriate.
8. Handle invalid and unexpected data.
9. Organize the final solution into ES modules where required.
10. Demonstrate programs against normal, boundary, and invalid test cases.

Task 1 - Admission Decision and Data Sanitization Engine

The admissions office receives applicant information from an external form. Since form values normally arrive as strings, the data cannot be trusted.

Example input:

```
const applicants = [
  {
    id: 'A101',
    name: 'Sara Khan',
    age: '19',
    matricMarks: '91',
    intermediateMarks: '87',
    entryTest: '78',
    program: 'BSCS',
    feePaid: 'true'
  },
  {
    id: 'A102',
    name: 'Ahmed Ali',
    age: 'twenty',
    matricMarks: '88',
    intermediateMarks: '',
    entryTest: '72',
    program: 'BSSE',
    feePaid: 'false'
  },
  {
    id: 'A103',
    name: 'Hassan',
    age: '18',
    matricMarks: '76',
    intermediateMarks: '83',
    entryTest: '91',
    program: 'BSAI',
    feePaid: 'true'
  }
];
```

Develop an admission-processing system. For every applicant, the system must:

- convert required numeric values explicitly;
- identify missing values and invalid numeric conversions;
- ensure all marks are between 0 and 100;
- verify age is between 17 and 30;
- reject unsupported programs;
- determine whether the application is valid before calculating merit;
- calculate merit using Matric 20%, Intermediate 30%, and Entry Test 50%.

An applicant is eligible only if Merit >= 70, Entry Test >= 60, and Intermediate >= 60.

Assign one of the following statuses:

```
SELECTED
WAITING
REJECTED
INVALID DATA
```

You must define sensible conditions distinguishing SELECTED and WAITING.

Additional Challenge

Produce a final summary containing:

- number of valid applicants;
- number of invalid applicants;
- highest merit applicant;
- lowest merit among valid applicants;
- average merit;
- number of applicants for each program.

Expected concepts: variables and primitive values, explicit type conversion, truthy/falsy behavior, arithmetic and comparison operators, logical operators, if/else, switch, ternary expressions, loops, functions, guard clauses, and scope.

Task 2 - Academic Performance Intelligence Engine

The Examination Department provides the following type of data:

```
const results = [
  { id: 1, name: 'Ali', department: 'CS', course: 'FSWD', marks: 82 },
  { id: 2, name: 'Sara', department: 'CS', course: 'FSWD', marks: 91 },
  { id: 3, name: 'Usman', department: 'SE', course: 'FSWD', marks: 47 },
  { id: 4, name: 'Hina', department: 'AI', course: 'FSWD', marks: 73 },
  { id: 5, name: 'Ahmed', department: 'SE', course: 'FSWD', marks: 64 },
  { id: 6, name: 'Zoya', department: 'AI', course: 'FSWD', marks: 91 }
];
```

A. Result Processing

Determine grade, status, and gradePoints for each student using the following grading policy:

```
85-100   A
80-84    A-
75-79    B+
70-74    B
65-69    B-
60-64    C+
50-59    C
Below 50 F
```

B. Ranked Merit List

Produce a new array containing students sorted by marks descending; if marks are equal, sort by name alphabetically. The original results array must remain unchanged.

C. Department Statistics

Produce an object similar to:

```
{
  CS: {
```

```
    students: 2,
    average: ...,
    highest: ...,
    passed: ...
  },
  SE: {
    ...
  }
}
```

The code must work even if a completely new department such as "DS" appears later.

D. Academic Queries

- first student scoring more than 85;
- index of the first failed student;
- whether at least one student failed;
- whether every student has marks within the valid range;
- all students scoring above the overall average;
- names of the top three students.

Restriction: For the main analytics operations, appropriate array methods should be used rather than implementing everything through traditional loops.

Expected methods: forEach(), map(), filter(), find(), findIndex(), reduce(), some(), every(), sort(), and slice(). Students must account for methods that mutate arrays.

Task 3 - Safe Student Profile Update Service

CampusFlow stores a student as:

```
const student = {
  id: 101,
  name: 'Ali',
  marks: 0,
  scholarship: false,
  address: {
    city: 'Islamabad',
    sector: 'F-10'
  },
  contact: {
    email: 'ali@university.edu'
  }
};
```

A profile update request may contain only some fields:

```
const update = {
  name: 'Ali Khan',
  address: {
    city: 'Rawalpindi'
  },
  contact: {
    phone: '03001234567'
  }
};
```

Implement a profile update service that returns a new student object. The original object must not be modified.

- properties not present in the update must remain unchanged;
- address.city should change while address.sector remains;
- existing contact.email should remain and contact.phone should be added;
- valid values such as 0, false, and an empty string must not automatically be replaced with defaults.

The task must appropriately demonstrate object destructuring, nested destructuring, renaming during destructuring, default values, array destructuring, rest parameters, object rest, spread operator, optional chaining, nullish coalescing, and property shorthand.

Reference Challenge

After generating the updated object, prove through code whether each of the following references is equal or different, and explain why:

- original object vs updated object;
- original address vs updated address;
- original contact vs updated contact.

Task 4 - University User Hierarchy and Dynamic this

CampusFlow contains multiple kinds of users. Create the following hierarchy:

```
Person
  |
  +---- Student
  |
  +---- FacultyMember
```

Person should contain at least name, email, and introduce().

Student should additionally contain rollNo, marks, introduce(), and getResult().

FacultyMember should additionally contain employeeId, department, and introduce().

Both subclasses must override introduce() but should reuse appropriate parent functionality.

Create several Student and FacultyMember objects, store them in the same array, and execute:

```
users.forEach(user => console.log(user.introduce()));
```

The correct version of introduce() should execute according to the actual object.

this, call() and bind() Challenge

Consider a guest speaker:

```
const guest = {
  name: 'Dr. Ahmed',
  email: 'ahmed@example.com'
};
```

- Use call() to execute the Person introduction method with guest acting as this.
- Detach a student method using const fn = student.introduce; and investigate what happens when fn() is invoked independently.
- Use bind() to create a safely bound version.

Students must demonstrate and explain constructor, new, this, extends, super, method overriding, runtime polymorphism, instanceof, call(), bind(), and detached methods. No closure-based solution may be used.

Task 5 - Modular Transcript Processing and Fault-Tolerant JSON

The registrar wants the transcript system separated into modules. Use at least the following files:

```
main.js
Student.js
gradeUtils.js
transcriptService.js
```

Student.js

Export the Student class as the default export.

gradeUtils.js

Provide named exports for functionality such as calculateGrade(), calculateAverage(), and calculateGPA().

transcriptService.js

Provide functionality for processing JSON transcript data. Example incoming JSON:

```
const rawData = `
[
  {
    "id": 101,
    "name": "Ali",
```

```
      "courses": [
        { "code": "CS301", "marks": 81 },
        { "code": "CS302", "marks": 74 }
      ]
    }
  ]
  `;
```

The service must:

- parse JSON safely;
- reject malformed JSON;
- reject missing student IDs;
- reject duplicate IDs;
- reject course marks below 0 or above 100;
- reject missing course codes;
- calculate each student's average and GPA;
- generate a valid JavaScript report object;
- convert the final report back to JSON.

Use meaningful error types such as Error, TypeError, and RangeError where appropriate. The caller must demonstrate try, catch, and finally. Use both named imports and default imports, and alias at least one named import. Students must not place all code into one file.

Task 6 - Legacy Course Registration Service Using Callbacks

CampusFlow still communicates with an old registration system that uses asynchronous callbacks. The registration sequence is:

```
Load Student
     |
Check Prerequisites
     |
Check Available Seats
     |
Register Course
```

Implement these four asynchronous operations using setTimeout() and the error-first callback convention:

```
callback(error, value)
```

Possible failures include:

- Student does not exist;
- prerequisite not completed;
- course is full;
- student already registered.

The system must stop immediately when any stage fails. For success, the final output should contain a registration confirmation object similar to:

```
{
  studentId: 101,
  course: 'CS401',
  status: 'REGISTERED'
}
```

Important requirements:

- each asynchronous operation must call its callback exactly once;
- return errors through the callback;
- do not use Promises;
- do not use async/await;
- correctly handle every asynchronous boundary.

Students should explain why this type of dependent asynchronous workflow becomes difficult to maintain as the number of operations increases. The explanation should focus on callback nesting and error handling, not closures.

Task 7 - Promise-Based Multi-Service Registration Engine (12 Marks)

The university decides to modernize the registration system. Create Promise-based versions of services such as:

```
getStudent()
getCourse()
checkPrerequisites()
checkSeats()
registerStudent()
sendEmail()
sendSMS()
writeAuditLog()
```

Part A - Registration Chain

Implement the following workflow using a Promise chain:

```
getStudent -> getCourse -> checkPrerequisites -> checkSeats -> registerStudent
```

The chain must correctly propagate data from one stage to the next. A missing return from a .then() callback should break the workflow, so students must structure the chain correctly. Use .then(), .catch(), and .finally() appropriately.

Part B - Post-Registration Services

After successful registration, sendEmail, sendSMS, and writeAuditLog start independently. Use an appropriate Promise combinator to determine the status of all three operations even if one fails. Produce a report similar to:

```
[
  { service: 'Email', status: 'fulfilled' },
  { service: 'SMS', status: 'rejected' },
  { service: 'Audit', status: 'fulfilled' }
]
```

Part C - Service Selection

Assume the system can retrieve course information from two independent servers. Use a suitable Promise combinator when CampusFlow should accept the first successfully returned result, rather than simply the first settled result. Students must justify their choice.

Expected concepts: Promise creation, resolve(), reject(), then(), catch(), finally(), Promise.all(), Promise.allSettled(), Promise.race(), Promise.any(), and Promise chaining. Students should choose combinators appropriately rather than using each one unnecessarily.

Task 8 - Concurrent Academic API Aggregator with async/await (12 Marks)

The final CampusFlow dashboard requires data from three services:

```
/users
/courses
/results
```

Implement:

```
async function getJSON(url, fetchImplementation)
```

The function must:

1. call fetchImplementation(url);
2. check response.ok;
3. throw an error for unsuccessful HTTP status codes;
4. parse and return JSON;
5. correctly propagate network and JSON parsing failures.

For reproducible lab testing, students may implement an asynchronous fakeFetch() that behaves like the real Fetch API and returns response-shaped objects. It must be capable of simulating 200 success, 404 response, 500 response, network failure, invalid JSON, and a slow response.

Concurrent Dashboard Loading

The three independent API calls must begin concurrently. Do not write sequential awaits when the operations do not depend on each other. Use an appropriate concurrent Promise strategy.

After obtaining the data, generate:

- total students;
- total courses;
- overall average marks;
- highest-performing student;
- number of failed students;
- department-wise statistics.

Timeout Challenge

Implement:

```
fetchWithTimeout(url, milliseconds)
```

Use AbortController, setTimeout(), clearTimeout(), and try/finally. The timer must be cleaned up whether the request succeeds or fails. The program should distinguish, where possible, between HTTP failure, network failure, invalid JSON, and timeout/abort.

Expected concepts: async functions, await, Promises, fetch, response.ok, response.json(), try/catch/finally, Promise.all(), AbortController, and concurrent asynchronous execution.